

Hajee Mohammad Danesh Science and Technology University

MSc in CSE

Dept. of Computer Science

Course: Software Project Management

Course Code: CSE 531

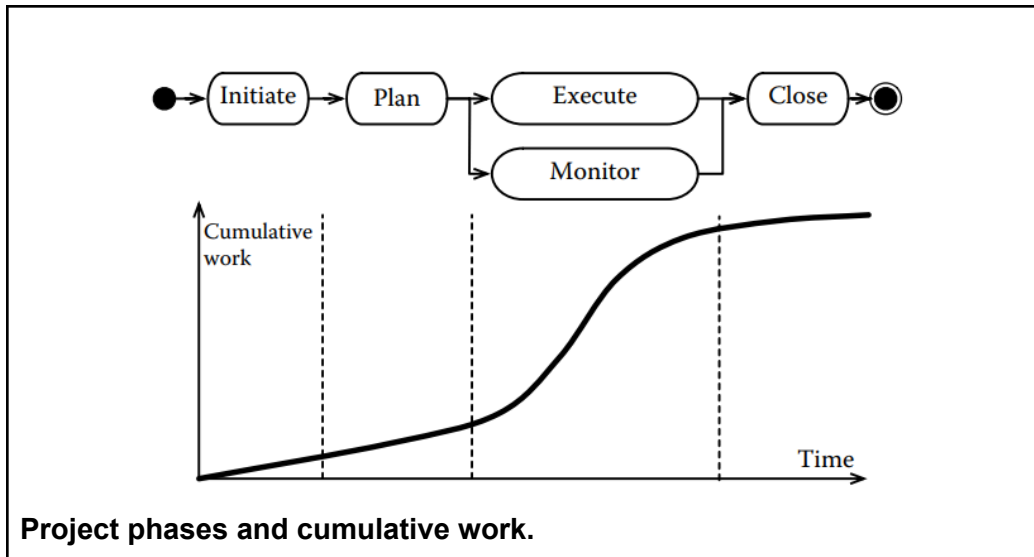
Prepared by: Professor Dr. Ashis Kumar Mandal

Reference Books: 1. Introduction to Software Project Management by Adolfo Villafiorita
 2. Software Engineering by Ian Sommerville

1.1 What is a Project

Project Management Institute (2004) defines a project as a temporary endeavor undertaken to create a unique product or service.

- First characteristic is that a project is temporary, that is, it has a beginning and an end.
 - An **initiating phase**, during which the project infrastructure and the project's goals are drafted.
 - A **planning phase**, during which project goals are refined, activities identified and scheduled, and many other support activities are properly planned.
 - An **executing phase**, during which the actual work takes place. Running in parallel, a monitoring phase measures the progress and raises flags when plans and reality disagree
 - A **final closing phase**, where the project outputs are handed out and the project is closed.
- The second characteristic is that a project delivers an output in the form of a product, a service, or a capability. The outputs are tangible, and often their properties are also measurable.
- The third characteristic is that projects are resource constrained. A limited time is available to build the project outputs. Also limited will be other project resources, such as the budget and the team
- The fourth characteristic is that a project requires a progressive elaboration to build the project outputs. This technique involves incrementally detailing and refining a project plan as more information becomes available throughout the project's lifecycle.
- The fifth and final characteristic is that a project delivers a unique output. Thus, what a project delivers has some novelty, one way or the other. This allows us to introduce the last important characteristic, namely, that a project always has some risk coming in the form of menaces or opportunities.



- Projects are focused on delivering specific outputs within defined constraints.
- Programs manage related projects to achieve broader organizational benefits.
- Portfolio: A collection of projects and programs that are managed and coordinated together to achieve strategic business objectives. A project portfolio typically includes:
 - Projects – individual efforts with specific goals (e.g., developing a new app).
 - Programs – groups of related projects (e.g., digital transformation initiative).
 - Resources – people, money, and tools shared across projects.
 - Performance metrics – data on cost, schedule, risk, and return.

1.2 Software project

A **software project** is a planned and organized effort to create, develop, maintain, or improve a software product or system. It involves the coordination of various tasks, resources, and people to achieve specific goals, usually within a set timeline and budget.

Managing Projects Project Manager and Project Stakeholder are two key roles in a software project

1.2.1 Project Manager (PM): The Project Manager is responsible for planning, executing, and overseeing the software project to ensure it meets its objectives within the defined scope, budget, and timeline.

Key Responsibilities:

- **Project Planning:** Define the project scope, create a detailed project plan, and allocate resources.
- **Execution and Monitoring:** Oversee day-to-day activities, manage the project team, track progress, and ensure tasks are completed on time.

- **Communication:** Serve as the main point of contact for the team, stakeholders, and clients, ensuring clear communication across all parties.
- **Risk Management:** Identify potential risks, resolve issues, and make adjustments to keep the project on track.
- **Budgeting and Scheduling:** Manage the project's budget, schedule, and ensure the project stays within these limits.

1.2.2 A Project Stakeholder is any individual, group, or organization that has an interest in the project's outcome or is affected by it. Stakeholders can be internal or external to the organization.

Key Types of Stakeholders:

- **Clients/Customers:** Those who will use or benefit from the software product.
- **End-users:** Individuals who directly interact with the software and provide feedback on functionality and usability.
- **Project Sponsor:** A key decision-maker who provides financial resources and support for the project.
- **Team Members:** Developers, testers, designers, and other contributors involved in building the software.
- **Regulators:** Entities that ensure the project complies with legal or industry standards.

Stakeholder Responsibilities:

- **Provide Requirements:** Contribute to defining the project's goals, features, and expectations.
- **Feedback and Approval:** Review progress, give feedback, and approve key deliverables.
- **Support and Influence:** Influence project direction and decision-making through involvement and feedback.

1.3 Software Project Management is the discipline of planning, organizing, executing, and overseeing software projects to achieve specific objectives within defined constraints, such as time, budget, and resources. It involves applying project management principles to the development and maintenance of software systems, ensuring that the project meets its goals in terms of functionality, quality, and user satisfaction.

Key Elements of Project Management

- **Scope management,** which includes all the activities to ensure that a project delivers according to the goals it sets.
- **Time management,** which defines a schedule for a project and delivers according to the schedule.
- **Cost management,** which defines a budget and controls spending during a project.

- **Change and configuration management**, which help manage changes in a project, while maintaining a coherent view on its outputs. These processes interact with all software development activities and also influence goals, schedules, and costs.
- **Risk management**, the set of activities to effectively manage menaces and opportunities. Similar to the previous case, risk management runs throughout a project, reducing the influence of unexpected events.
- **Quality management**, the set of activities to ensure that a project defines quality goals and delivers accordingly the goals.
- **Human resource management and stakeholder management: Human Resource Management** focuses on efficiently managing the project team to optimize performance, while **Stakeholder Management** involves engaging and aligning with individuals or groups affected by or influencing the project to meet their expectations.

2. Software Development Activities and Their Organization

Software development involves a series of structured activities aimed at building, delivering, and maintaining high-quality software. These activities are usually organized into a Software Development Life Cycle (SDLC). Software development activities include requirement analysis, design, implementation, testing, deployment, and maintenance

2.1 Software Requirements Definition includes the methods to identify and describe the features of the system to be built. The main output of this activity is one or more artifacts describing the (software) requirements of a system, namely, the functions it has to implement and the other properties it has to have.

There are two main output formats for these artifacts, **textual or diagrammatic**.

2.1.1 Textual:

- When the textual format is used, the requirements are written in English, in some cases using a restricted set of language or predefined linguistic patterns.
- For instance, special words, such as “shall,” might be required to indicate an essential requirement..
- Another very common representation writes requirements in the form of user stories, using the following pattern:

As a [user] I want to do [this] because of [that].

The advantage of this approach is that each requirement clearly identifies the user, the function that has to be performed, and the motivation for the function to be implemented, something that helps identify the priority or importance of a requirement.

2.1.2 Diagrammatic. The diagrammatic notation describes requirements with a mix of diagrams and textual descriptions. Diagrams depict the interaction between the user and the system and the textual description explains the interaction using a sequence of steps.

The most common graphical notation is that of the use case diagrams of UML and the corresponding textual descriptions are called use cases.

2.1.3 activities

The requirement engineering discipline includes the activities necessary to define and maintain requirements over time. Simplifying a bit, requirements engineering entails a cyclical refinement process in which the following steps are repeated at increasing levels of detail till a satisfactory level of know-how about a system is achieved.

In more detail, the process is composed of four steps:

- Requirements elicitation
- Requirements structuring
- User experience design
- Requirements validation.

2.1.3.1 Requirements Elicitation

Requirements elicitation is the activity during which the list of features of a system are elicited from the customer. This activity can be performed with interviews, workshops, or the analysis of existing documents.

2.1.3.2 Requirements Structuring

Requirements structuring is the phase during which the requirements are annotated to make their management and maintenance simpler. During the process, requirements are

- **Isolated and made identifiable.** Each requirement is clearly isolated and distinguished from the others and is also assigned a unique identifier.
- **Organized and classified.**
 - A simple classification distinguishes between functional and nonfunctional requirements.
 - Functional requirements describing what the system has to do.
 - Non Functional are requirements describing what other properties the system should exhibit (e.g., “the system will have to run on Windows devices”). Nonfunctional requirements define the quality and performance standards of a software system — ensuring it is usable, reliable, secure, and maintainable.
 - Functional requirements are usually organized in functional areas. Each functional area groups requirements describing a homogeneous set of functions. For instance, a requirement document might have an “accounting functions” section describing all requirements pertaining to accounting functions.
 - Nonfunctional requirements are often organized in four groups: usability, reliability, performance, and supportability.
- **Annotated.** Requirements are annotated to simplify their management and to support planning activities, like, for instance, which requirements should be implemented first. It is a good practice to assign each requirement at least two properties, namely, the importance for the customer and the difficulty to develop, for instance, using values from 1 to 5.

Here, it is sufficient to mention that requirements are often annotated with Traceability information, which has the goal of highlighting where a requirement originates from.

- **Traceability** shows the relationships among requirements and the relationships among requirements and other artifacts of software development. This allows one to understand the impact of changes.
- **History log**, which records the changes each requirement has undergone. The history log traces how requirements have changed over time.

2.1.3..3 User Experience Design

User experience design has the goal of providing a coherent and satisfying experience on the different artifacts that constitute a software system, including its design, interface, interaction, and manuals.

The typical user experience design activities include

- **User-centered analysis**, which has the goal of understanding how users will interact with the system. User-centered analysis, which has the goal of understanding how users will interact with the system. It runs in parallel with the requirements definition and requires the organization of workshops and other activities (e.g., surveys) to profile the users, analyze which tasks they will perform, and define which style guides will be followed in designing the system.
- **User-centered design**, which has the goal of specifying how users will actually interact with the system. It runs in parallel with the requirements definition and system design. The outputs include storyboards describing the interaction, mock-ups, and prototypes. (A mock-up is a fullsize model of something that has not yet been built, showing how it will look or operate.

2.1.3.4 Requirements Validation

Requirements validation is the phase during which the requirements are analyzed to find

- Inconsistencies,
- Incompleteness,
- Duplicates,

Some techniques can be used to validate requirements.

- **Inspections**: are based on the work of a team that analyzes the content of documents and highlights any issue.
- **Formal analyses**. use mathematical notations (such as first-order logic) to represent requirements and automated tools (such as theorem provers and model checkers) to prove properties about the requirements.

2.2Design : The design phase translates the requirements gathered during the analysis phase into a blueprint or plan for building the software.

Types of System Design

- **High-Level Design (HLD)**: Focuses on the system architecture and technology choices.

- **Low-Level Design (LLD):** Focuses on the detailed implementation of components and data structures.

2.2.1 High-Level Design (HLD):

Architecture Design: It is High level design that defines the overall structure of the system. Determines the major components (e.g., client, server, database) and how they interact.

Common Patterns:

- **Monolithic architecture:** A monolithic architecture is a unified model where all components of an application are interconnected and interdependent, forming a single, cohesive unit. The user interface, business logic, and data access layers are all combined into one program on a single platform.
- **Client-Server architecture:** This is a distributed model that separates the provider of a service (the server) from the consumer (the client). The client requests resources or services, and the server fulfills those requests. Communication happens over a network.
- **Microservices architecture:** An architectural style that structures an application as a collection of small, independent, loosely coupled services. Each microservice is a mini-application that owns its own data and implements a specific business capability. They communicate via lightweight protocols like HTTP/REST or messaging.
- **Event-driven architecture:** An architecture pattern that promotes the production, detection, consumption of, and reaction to events. System components communicate and coordinate through events rather than direct calls.
- **Microkernel / Plugin architecture:** Also known as a plug-in architecture, this pattern consists of a core system (the microkernel) that provides the fundamental functionality and a set of plug-in modules that extend this core with additional features. The kernel is kept minimal and stable.
- **Layered (N-tier) architecture:** The application is organized into horizontal layers, each with a specific role. Each layer has a specific responsibility and only interacts with the layer directly above or below it. The most common is the 3-tier architecture.

Database Design: Selecting the type of database (SQL vs. NoSQL) and defining a high-level schema.

Technology Stack Selection: Choosing programming languages, frameworks, libraries, and tools.

Component Diagramming: Identifying major modules (e.g., User Management, Payment Processing, Reporting) and how they interact with each other.

Communication Flow: Defining how different parts of the system will communicate (e.g., APIs, message queues).

2.2.2 Low-Level Design (LLD):

- **Detailed Class/Module Design:** Defining specific classes, their attributes, methods, and relationships (using UML diagrams like Class Diagrams).
- **Data Structures and Algorithm Design:** Specifying the exact logic and algorithms for complex functions.
- **User Interface (UI/UX) Design:** Creating detailed wireframes, mockups, and interactive prototypes.
- **API Specification:** Detailing each API endpoint, its request/response structure, and error codes (often using tools like Swagger/OpenAPI).
- **Security Design:** Outlining authentication, authorization, encryption, and other security measures.

Diagrams can be used both in High-Level Design (HLD) and Low-Level Design (LLD) depending on the diagram type.

High-Level Design (HLD) Diagrams

- 1 Component Diagram - Shows major components/services and their dependencies
- 2 Deployment Diagram - Shows physical hardware and software deployment
- 3 Data Flow Diagram (DFD) - Shows how data moves through the system
- 4 System Context Diagram - Shows system boundaries and external interfaces
- 5 Architecture Diagram - High-level technology stack and connections
- 6 Use Case Diagram - User-system interactions
- 7 Activity Diagram - behavioral diagram that represents the flow of control from one activity to another

Low-Level Design (LLD) Diagrams

- 1 Class Diagram - Detailed classes, attributes, methods, relationships
- 2 Sequence Diagram - Show object interactions over time
- 3 ER diagram/Relational Tables
- 4 Activity Diagram
- 5 Flowcharts - Detailed algorithm logic

2.3 Implementation

The goal of the implementation phase is to translate the detailed design documents into a functional, efficient, and well-structured software product in the chosen programming language(s).

The goal of the implementation phase is writing the code implementing the components individuated in the architecture.

Key Activities

- **Setting up the Development Environment:** Prepare and configure all necessary tools, IDEs, and version control systems for coding.

- **Coding:** Developers write source code based on design documents, following coding standards.
- **Unit Testing:** Test each individual module immediately after coding to verify it works correctly.
- **Code Integration:** Combine all modules and test their interaction to ensure the system functions as a whole.
- **Code Review and Optimization:** Review code for quality, fix issues, and improve performance and security.

Some of the aspects of this activity that are more closely related to project management include

- **Collection of productivity and size metrics,** which allow one to measure the speed at which code is delivered and the amount of work that has been performed.
- **Collection of quality metrics,** which allow one to measure the quality of the system to be developed and trends in the development process.
- **The use of coding standards,** which are guidelines describing best practices and the preferred styles to write code.

Verification and Validation

validation ensures that we are building the right system , while verification ensures that we built the system right (i.e., it checks if the product meets the specified requirements)..

Verification: Verification is the process of evaluating a system or its components to determine whether the product of a given development phase meets the specified requirements. It answers the question, "**Are we building the product right?**" This process focuses on ensuring that the system or software conforms to its design, specifications, and standards. Common activities include inspections, reviews, and testing individual components.

Validation: Validation is the process of evaluating a system or component during or at the end of the development process to determine whether it meets the user needs and intended use. It answers the question, "**Are we building the right product?**" This process ensures that the final product satisfies the user requirements and functions in a real-world environment. Activities include user acceptance testing (UAT), system testing, and field testing.

Verification and validation are collectively known by the acronym V&V.

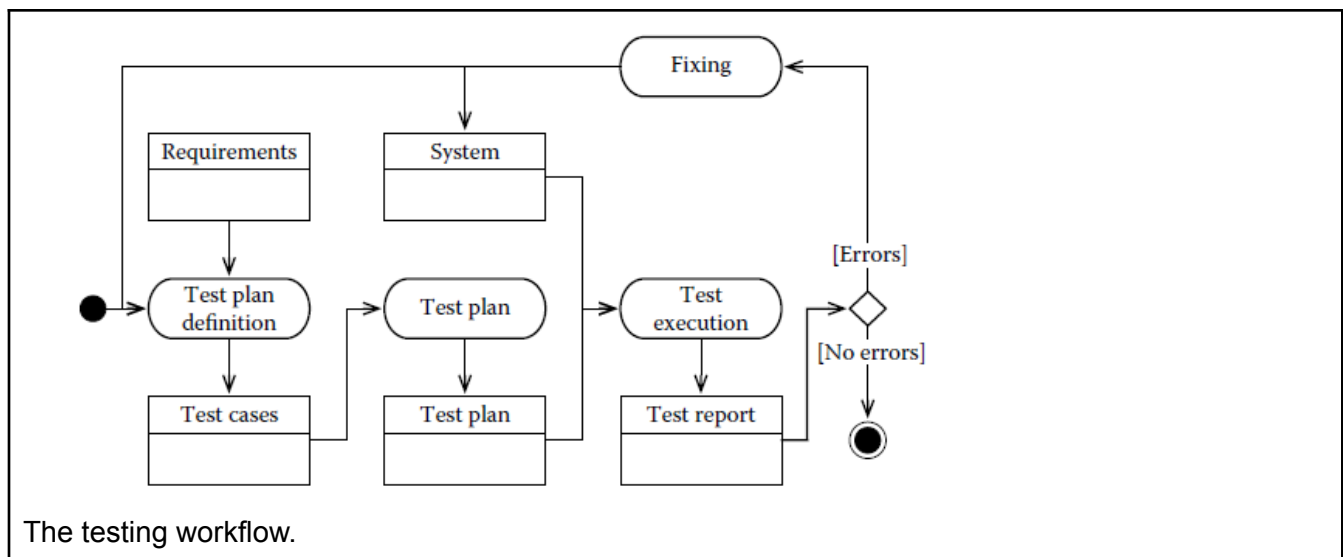
2.4 Testing Testing is one way of performing verification and validation. Other methodologies include simulation, formal validation, and inspections.

Testing activities can be classified according to their scope.

- **Unit testing,** when the goal is to verify the behavior of a piece of code, such as a class. Unit testing verifies that the code under investigation behaves as specified by the system

architecture. The execution of unit tests can be easily automated, since they can be written as pieces of code.

- **Integration testing**, when the goal of the testing activity is to ensure that the components of a system behave as expected when they are assembled. Integration testing looks for inconsistencies in the way data are exchanged between components.
- **System testing**, when the goal of the testing activity is to ensure that the system behaves as expected and correctly implements all the requirements.
- **Usability testing**, running in parallel with the other testing activities, has the goal of verifying whether the user experience and interaction are intuitive, effective, and satisfying. Usability testing is particularly relevant in designing user interfaces for safety-critical systems to reduce the probability of human errors.

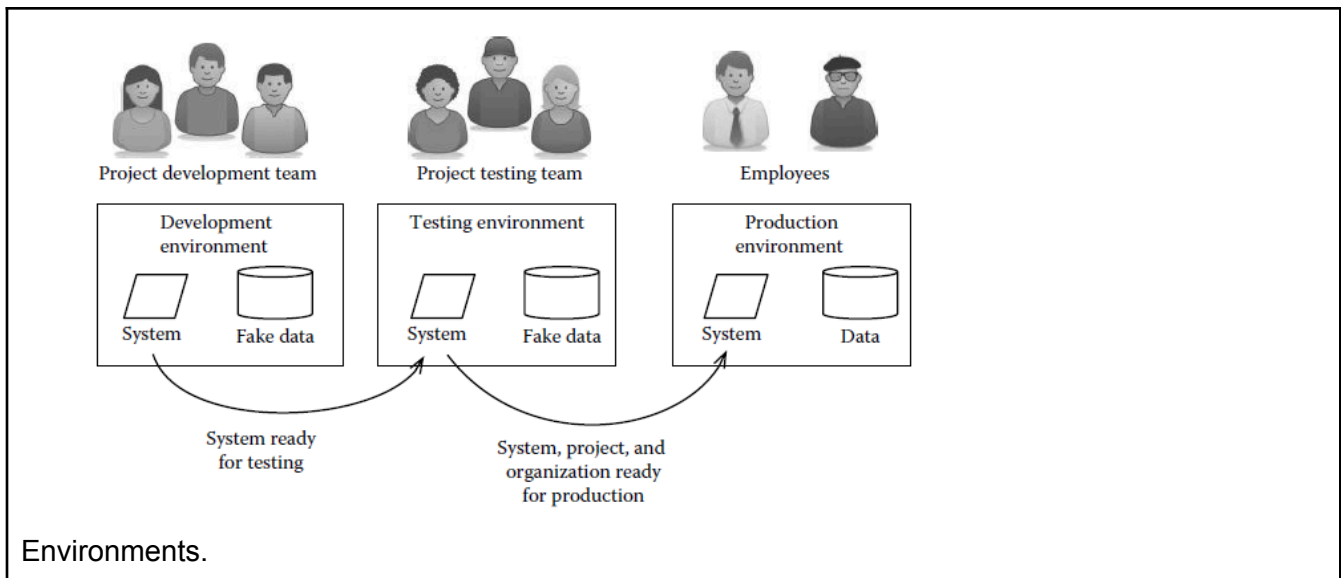


2.5 Deployment

The final step of the development process is releasing and installing a system so that it can be used by the customer and operations start. The transition to operations can be very simple for the project team.

A standard practice for projects of this kind makes sure that any change or software evolution does not interfere with production. To achieve this goal, the project team sets up three exact and independent replicas of the same operating environment, as shown in Figure

- A development environment, where the actual development of the software takes place.
- A testing environment, where the team tests a system that is ready for deployment.
- A production environment, where the system is actually used



Three factors need to be taken into account when deploying a new system.

They are

1. **The human factor:** are the people ready to use the system?
2. **The data factor:** are all the data that are needed for the system to run available to the new software?
3. **The hardware factor:** are all interfaces ready and functional?

Migration strategy, which defines an approach to the introduction of the new system.

- **Cut-over**, when the old system is replaced by the new one.
- **Parallel approach**, when the old and the new systems operate simultaneously for a period. This allows the new system to be tested and evaluated before the actual switch takes place.
- **Piloting**, when the system is installed for a limited number of users or for a specific business unit. This approach reduces the burden to users (who do not have to live with two systems), but it maintains the complexity of having two environments—the old and the new systems—both alive.
- **Phased approach**, when functions are rolled out incrementally.

The final step is the implementation of the release process, which in turn consists of the following steps:

- **Deliver training**, to ensure that the users acquire the necessary skills to use the new system.
- **Perform data migration**, which includes updating the data used in the production environment so that it can be used by the new system. This is a delicate step, which requires a thorough testing of the migration scripts and a backup of the existing data structures.
- **Install the new system**, which puts the new system in production.

- **Set up the support infrastructure**, namely, set up an infrastructure to support operations. More on this in the next section.

2.6 Operations and Maintenance

Operations and maintenance include the activities to ensure that a product remains functional after its release.

Supporting and Monitoring Operations

- **Providing technical support.** The support is meant to help users get acquainted with the system and it can be organized as a help-desk collecting tickets from users. Some of these tickets are requests for clarifications on the use of the system. Others will signal malfunctions, glitches, and requests for improvement, triggering maintenance activities. See the next section for more details.
- **System monitoring.** A set of metrics might be collected on the system after its initial release to monitor performances, issues, and other system features.

Maintenance

Maintenance occurs throughout the lifecycle of a system, before it is retired. It can be framed either as a project or operational work and, as such, it often poses a dilemma to the project manager.

Four categories of maintenance for software:

- **Corrective:** Fixing problems that were found after the software was released.
 - **Preventive:** Fixing potential problems before they happen or are noticed by users.
 - **Adaptive:** Updating the software to work with new changes, like making it compatible with a new version of an operating system.
 - **Perfective:** Improving the system's performance or adding features to make it better.
-